Dhuno: A Real-Time Synchronized Music Streaming Platform with Host-Controlled Listening Rooms

Project Report Submitted

in partial fulfillment of the requirements for the award of the

Bachelor of Technology

in

Computer Engineering

by

Nabin Agrawal [Roll No.]

Kumar Vanshaj [Roll No.]

Parlin Kant Goyal [Roll No.]

Under the supervision of

[Supervisor Name]



Department of Computer Science and Engineering

International Institute of Information Technology, Bhubaneswar

October 2026

Certificate

This is to certify that the report entitled "Dhuno: A Real-Time Synchronized Music Streaming Platform" submitted by Nabin Agrawal ([Roll No.]), Kumar Vanshaj ([Roll No.]) and Parlin Kant Goyal ([Roll No.]) to International Institute of Information Technology, Bhubaneswar is a record of bonafide project work under my supervision, and the report is submitted for the evaluation of the BTech Degree Final-Semester Project.

(Supervisor)
[Supervisor Name]

Declaration

We certify that

1. The work contained in the report has been done by ourselves under the general supervision of our supervisor.
2. The work has not been submitted to any other Institute for any degree or diploma.
3. We have followed the guidelines provided by the Institute in writing the report.
4. We have conformed to the norms and guidelines given in the Ethical Code of Conduct of the Institute.
5. Whenever we have used materials (data, theoretical analysis, and text) from other sources, we have given due credit to them by citing them in the text of the report and giving their details in the references.
6. Whenever we have quoted written materials from other sources, we have put them under quotation marks and given due credit to the sources by citing them and giving required details in the references.

Nabin Agrawal	**Kumar Vanshaj**	**Parlin Kant Goyal**
[Roll No.]	[Roll No.]	[Roll No.]

Acknowledgments

We would like to express our sincere gratitude to our respected supervisor, [Supervisor Name], for the invaluable guidance, support and encouragement given throughout this project. Their insights and expertise were instrumental in shaping our work and helped us overcome numerous challenges. We are also thankful to the faculty and staff of the Department of Computer Science and Engineering, International Institute of Information Technology, Bhubaneswar, for providing the resources and a conducive learning environment. Finally, we would like to thank our families and friends for their unwavering support and understanding during the course of this project.

Nabin Agrawal
[Roll No.]

Kumar Vanshaj
[Roll No.]

Parlin Kant Goyal
[Roll No.]

Abstract

Listening to music has become an individual, on-demand activity. Mainstream streaming services excel at personal playback, yet listening *together* in real time, with every participant hearing the same second of the same song, is either locked behind a specific ecosystem or missing altogether. Independent artists, meanwhile, lack a simple route to publish tracks and playlists directly to listeners.

This report presents **Dhuno**, a full-stack web platform that addresses both gaps. Listeners create or join *rooms* in which playback is synchronized across all participants through a server-authoritative state model: only the host (or a DJ delegated by the host) may play, pause, seek or skip, while every other participant follows. Around this core, Dhuno provides user accounts with JWT sessions, Google sign-in and e-mail based two-factor authentication, playlists, liked songs, search, location-aware room discovery, in-room chat and reactions, and an artist dashboard for uploading music and creating playlists.

The system is built on a React 19 and TypeScript frontend, a Node.js/Express 5 backend, MongoDB for durable data, Redis for ephemeral room state, caching and rate limiting, Socket.IO for real-time messaging, and Cloudinary for audio and image storage. The report states the problem, reviews existing platforms and the technical literature, sets out the planned architecture and synchronization protocol, and documents the implementation, deployment setup and planned improvements such as enhanced search, recommendations and clock-offset compensation.

Keywords: Music Streaming, Synchronized Playback, Real-Time Systems, WebSocket, Socket.IO, MERN Stack, Redis, Cloud Media Storage, Full-Stack Development.

Contents

Abstract **4**

1 Problem Statement **6**
1. Background . . . 6
2. Problem Definition . . . 6
3. Objectives . . . 6
4. Scope and Limitations . . . 7

2 Previous Work **8**
1. Existing Music Streaming Platforms . . . 8
2. Technical Foundations from the Literature . . . 8
3. Limitations of Existing Solutions . . . 9
4. Research Gap . . . 9

3 Planned Work **10**
1. Proposed Solution . . . 10
2. System Architecture . . . 10
3. Requirements . . . 10
4. Data Model Plan . . . 11
5. Synchronization Protocol Design . . . 11
6. Development Plan . . . 13
7. Planned Enhancements . . . 13
   - 7.1 Enhanced Search . . . 13
   - 7.2 Music Recommendations . . . 13
   - 7.3 Performance Improvements . . . 13

4 Implementation **14**
1. Technology Stack . . . 14
2. Frontend . . . 14
   - 2.1 Structure, Player and Contexts . . . 14
   - 2.2 Live Session Page . . . 15
   - 2.3 Artist Workspace and Offline Use . . . 15
3. Backend and Database . . . 15
4. Authentication and Security . . . 16
   - 4.1 Sessions and Passwords . . . 16
   - 4.2 Google Sign-In, Two-Factor Authentication and Recovery . . . 16
5. Cloud Media Storage . . . 17
6. Real-Time Features . . . 17
7. Caching and Performance . . . 18
8. Deployment and Maintenance . . . 19
9. Verification and Test Scenarios . . . 19

5 Conclusion and Future Work **20**
1. Conclusion . . . 20
2. Future Work . . . 20

Bibliography **21**

Chapter 1

Problem Statement

This chapter introduces the context in which Dhuno was conceived, states the problem it sets out to solve in precise terms, and lists the objectives and boundaries of the work. The remaining chapters review existing work (Chapter 2), present the planned solution (Chapter 3), document the implementation (Chapter 4) and conclude (Chapter 5).

1. Background

Music consumption has moved almost entirely from physical media and broadcast radio to on-demand streaming. A listener can reach millions of tracks from a phone or a browser, build personal playlists and resume exactly where they stopped. This convenience has, however, made listening a largely *solitary* activity. People who are in different places have no simple way to hear the same song at the same moment short of a video call and a phone held near a speaker.

Shared listening is a demanding distributed-systems problem. Every participant's device has its own audio buffer, network latency and clock, so if each person simply presses *play* the streams drift apart by seconds. A usable solution must (i) keep one authoritative description of what is playing and at which position, (ii) push every change to all participants with low delay, and (iii) let a participant who joins late, or whose connection drops briefly, converge to the same position without disturbing everyone else.

A second gap exists on the supply side. Independent artists want a direct way to publish tracks, cover art and playlists and to see how they are played, whereas general-purpose services place this behind separate artist tooling. Dhuno is a web-based platform designed around both observations: it combines an ordinary streaming experience (play, pause, playlists, likes, search) with *listening rooms* in which playback is kept in lock-step across participants, and with an artist area where creators upload and organize their own music.

2. Problem Definition

The problem addressed by this project is stated formally below.

> *To design and implement a web-based music streaming platform in which multiple users can join a listening room and hear the same track at the same playback position in real time, with playback control restricted to the room's host; and which additionally supports personal libraries (playlists and liked songs), music and room discovery, and an artist workspace for uploading music and creating playlists, all behind secure authentication.*

The statement decomposes into the sub-problems in Table 1.1; each corresponds to a design decision described in Chapters 3 and 4.

3. Objectives

O1. Build a responsive single-page application with one global audio player that survives page navigation.

Table 1.1: Sub-problems derived from the problem statement

ID	Sub-problem	Why it is difficult
P1	Playback synchronization	Clients differ in latency, buffering and clocks; "everyone presses play" drifts apart within seconds.
P2	Control authority	Only the host may change playback although all clients share one channel; authorization must be enforced on the server.
P3	Late join and reconnection	A newcomer or reconnecting listener must learn the song, position and queue immediately.
P4	Ephemeral versus durable state	Playback position changes continuously and must be fast; users, songs and playlists must be durable.
P5	Media ingestion and delivery	Audio and images must be validated, stored off the application server and delivered quickly.
P6	Secure identity	Accounts, sessions and password recovery must resist brute force and credential theft.
P7	Discovery	Users must find songs quickly and discover relevant rooms, for example by location.

O2. Implement real-time rooms in which play, pause, seek, skip and queue changes made by the host appear on every participant's device.

O3. Enforce host-only (or DJ-only) playback control on the server and provide a delegated *DJ mode*.

O4. Let listeners create playlists, like songs, search for music and see recently played tracks.

O5. Provide an artist workspace for uploading songs and cover art, creating playlists and viewing basic analytics.

O6. Keep audio and images in cloud media storage and only references in the database.

O7. Provide secure authentication: hashed passwords, token sessions, Google sign-in, optional two-factor authentication, password reset and rate limiting.

O8. Keep room state resilient to reconnection and restarts, and keep frequently read data fast through caching.

O9. Package the system so that it can be built, configured and deployed repeatably.

4. Scope and Limitations

In scope. Dhuno is delivered as a web application for modern desktop and mobile browsers. It covers user and artist accounts, the streaming player, playlists, likes, search, synchronized rooms with chat, reactions, queue management and vote-to-skip, location-based room discovery, notifications, artist clips, theming, offline downloads in the browser, and container-based deployment.

Out of scope. A licensed commercial catalogue (music is uploaded by artists who own the rights to it), digital rights management, native mobile apps, machine-learning recommendation, mood detection and voice control (planned work, Chapter 3), and sample-accurate audio alignment: Dhuno targets perceptually acceptable synchronization for casual group listening.

Chapter 2

Previous Work

This chapter surveys what has already been done in shared music listening and the established techniques on which a synchronized streaming platform depends. It ends with the limitations of existing solutions and the gap that Dhuno targets.

1. Existing Music Streaming Platforms

Mainstream Streaming Services

Large streaming services are built around individual listening: a personal library, personal recommendations and a private queue. Several have recently added group features. Spotify introduced *Jam* in September 2023, a real-time collaborative playlist that up to 32 people can control together [8]. Apple added *SharePlay* in late 2021, which lets people on a FaceTime call listen to Apple Music together [9]. These features show real demand for shared listening, but both are tied to the vendor's app, devices or call session, and neither lets a stranger open a link in a browser and join a themed room.

Social Listening Rooms

Before the major services added group features, *Turntable.fm* popularised online rooms in which users take turns as DJ while the others listen and vote [10]. The service was discontinued in 2013 but showed that room-based listening with chat and a rotating DJ is engaging; Dhuno adopts the room metaphor, and its optional DJ mode is inspired by this model.

Comparison

Table 2.1 compares these approaches with Dhuno, based on public announcements and documentation; features of commercial products may have changed since.

Table 2.1: Comparison of shared-listening approaches

Platform	Group listening model	Who controls playback	Access
Spotify Jam [8]	Shared, collaborative queue in a live session	Multiple participants (up to 32)	Inside the Spotify app
Apple SharePlay [9]	Listening together during a FaceTime call	Participants in the call	Apple devices
Turntable.fm [10]	Rooms with rotating DJs	The current DJ	Web (closed 2013)
Dhuno	Public, private and location-based synchronized rooms	Host, or a DJ delegated by the host	Any modern browser

2. Technical Foundations from the Literature

Real-time communication. Repeated polling wastes bandwidth and adds delay; the WebSocket protocol [1] instead provides a persistent full-duplex connection supported by all major

browsers. Socket.IO [11] builds on it and adds named rooms and broadcast, acknowledgements, heartbeats, automatic reconnection and a fallback transport. Dhuno uses Socket.IO for live room traffic and ordinary HTTP/REST [7] for everything else.

Clock and playback synchronization. Keeping devices at the same position is a clock-agreement problem. The Network Time Protocol [2] estimates the offset between a client and a server clock from four timestamps: the client's request time t_0, the server's receive and send times t_1 and t_2, and the client's receive time t_3:

$$\theta = \frac{(t_1 - t_0) + (t_2 - t_3)}{2}. \tag{2.1}$$

Cristian's algorithm [3] is a simpler variant assuming symmetric delay. If the server publishes the instant at which a song's position was zero, any client that knows its offset can compute its position.

Authentication. Stateless sessions are commonly implemented with JSON Web Tokens [4], which carry signed claims the server verifies without a database lookup. Passwords must be stored hashed; *bcrypt* [6] uses a per-password salt and an adjustable cost factor. Third-party sign-in such as "Sign in with Google" is based on OAuth 2.0 [5].

Data management and media. Document databases such as MongoDB suit nested, evolving structures such as songs, playlists and rooms, and their `2dsphere` indexes support queries such as "rooms within a given distance of a point" [12]. In-memory stores such as Redis [13] provide key expiry, sets and sorted sets, which fit cached results, room snapshots, presence and rate-limit counters. Cloud media services such as Cloudinary [14] accept uploads through an API and serve files through a CDN, so the application only keeps a URL and an identifier.

3. Limitations of Existing Solutions

G1. Ecosystem lock-in. Group listening in commercial services requires the vendor's app, account type or devices [8, 9].

G2. Weak host authority. Collaborative models give many participants control; a simple "one host controls the room" model is poorly supported.

G3. No local, community-oriented discovery. Rooms are formed by invitation; finding a public room near the user is not part of the experience.

G4. Fragile state. Rooms of older social-music services depended on a running server process; a restart or dropped connection could break the shared session.

G5. Separated listener and creator experiences. Uploading music needs separate tools, so independent artists cannot easily present music, playlists and live rooms in one place.

4. Research Gap

Together, G1–G5 define the gap this project fills: an open, browser-based platform with server-authoritative synchronized rooms, host control, location-aware discovery and an integrated artist workspace. Dhuno answers them as follows: G1 by browser-only access with e-mail or Google sign-in (Sections 4.2 and 4.4); G2 by host-only control checked on the server plus an optional DJ mode (Sections 3.5 and 4.6); G3 by public rooms with coordinates and a `2dsphere` index (Sections 3.4 and 4.3); G4 by room snapshots in Redis that are restored on reconnect or restart (Sections 4.6 and 4.7); and G5 by an artist role with upload, playlists, clips and analytics (Sections 4.2 and 4.5).

Chapter 3

Planned Work

This chapter describes the solution planned in response to the gaps identified in Chapter 2: the approach, the architecture, the requirements, the data model, the synchronization protocol, the development plan and the enhancements planned beyond the current release.

1. Proposed Solution

Dhuno is planned as a modular three-tier web application with a separate real-time channel. The listener's browser runs a React single-page application. A Node.js/Express server exposes a REST API for request/response work (accounts, songs, playlists, room management) and, through the same HTTP server, a Socket.IO gateway for live room events. MongoDB is the system of record, Redis holds transient state, and Cloudinary stores audio and images. Six principles guide the design:

D1. Server-authoritative playback. Clients send *intents* (play, pause, seek); the server validates them and broadcasts the resulting state.

D2. Host-only control. The check "may this user control playback?" lives on the server, so a modified client cannot bypass it.

D3. Split state. Fast-changing data (position, queue, presence) lives in memory with a Redis snapshot; durable data lives in MongoDB.

D4. One global player. A single audio element owned by a context provider keeps playback continuous during navigation.

D5. Artist-first content flow. Artists upload through the API directly to cloud storage; the database keeps only URLs and identifiers.

D6. Defence in depth. Input validation, hashed credentials, token checks on REST and sockets, and per-route rate limiting.

2. System Architecture

Figure 3.1 shows the planned architecture. The browser talks to the backend over HTTPS for REST calls and over a WebSocket for room events. Both share one process and the same data stores. External services are used through narrow interfaces: Cloudinary for media, an SMTP server for one-time codes and, as planned work, a BullMQ job queue for deferred tasks.

The **presentation layer** renders pages, owns the audio player and one shared socket connection. The **application layer** validates input, authenticates requests and applies business rules, and the socket gateway holds the authoritative in-memory room state. The **data layer** consists of MongoDB (durable documents), Redis (cache, snapshots, presence, rate limits) and Cloudinary (binary assets).

3. Requirements

The requirements in Table 3.1 were derived from the objectives in Chapter 1.

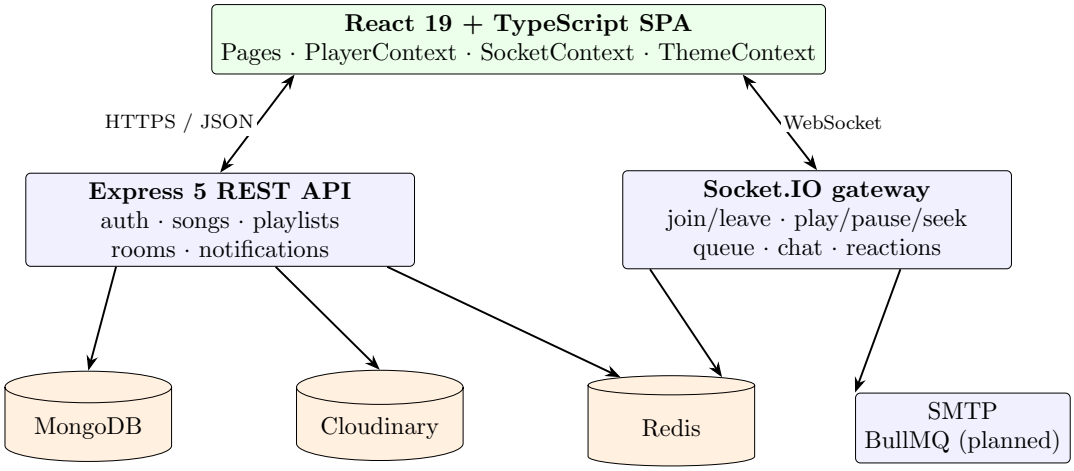


Figure 3.1: Planned system architecture of Dhuno

Table 3.1: Functional (FR) and non-functional (NFR) requirements

ID	Requirement
FR1	Users can sign up, log in and out, reset a forgotten password and edit their profile.
FR2	Users can play, pause, seek and skip, with a persistent mini player on every page.
FR3	Users can create playlists, add and re-order songs, like songs and view recently played tracks.
FR4	Users can search for songs and artists and see trending songs.
FR5	Users can create, discover (including by location), join and leave rooms.
FR6	In a room only the host (or assigned DJ) controls playback; all participants receive the same state.
FR7	Participants can chat, react, add songs to the queue and vote to skip, subject to room settings.
FR8	Artists can upload songs and cover art, create playlists and view analytics.
NFR1	Playback commands reach all participants with a delay imperceptible in casual listening.
NFR2	Late or reconnecting participants converge to the current room state automatically.
NFR3	Credentials are hashed, sessions expire and authentication endpoints are rate limited.
NFR4	Frequently read data is cached; uploads are limited in size and type; builds are reproducible in containers.

4. Data Model Plan

The entities and relationships were planned as in Figure 3.2. MongoDB was chosen because rooms, queues and playlists contain nested arrays that would need many join tables in a relational design. Rooms store a GeoJSON `Point` with a `2dsphere` index so that nearby rooms are found with a single query; a room also embeds its queue and a capped list of recent chat messages, so opening a room needs one read; and liked and recently played songs are identifier arrays on the user, which keeps the most frequent reads cheap.

5. Synchronization Protocol Design

Playback Timeline Model

Instead of broadcasting the position continuously, the server publishes one *reference instant* whenever playback starts or changes. If the host starts a song at position p_0 seconds at server

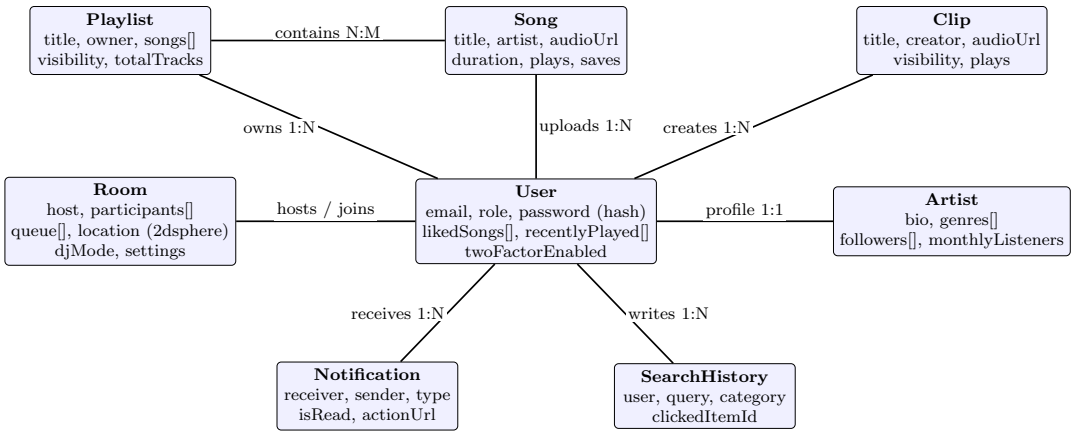


Figure 3.2: Planned entity–relationship overview (MongoDB collections)

time T_{now} (milliseconds), the server stores

$$T_{start} = T_{now} - 1000\, p_0, \tag{3.1}$$

the instant at which the song's position would have been zero. Any client that knows T_{start} computes the current position itself:

$$p(t) = \frac{t - T_{start}}{1000}. \tag{3.2}$$

A pause clears T_{start} and stores the paused position; a seek recomputes T_{start}. Network traffic is therefore proportional to host actions, not to elapsed time.

Event Protocol and Message Flow

The host sends intents (`room_play`, `room_pause`, `room_seek`, `room_next`); the server checks that the sender is the host or current DJ and broadcasts the new state; all other participants only receive these events. Chat, reactions, queue additions and skip votes are separate events with their own rules (Section 4.6). Figure 3.3 shows the flow for a play action and for a late join.

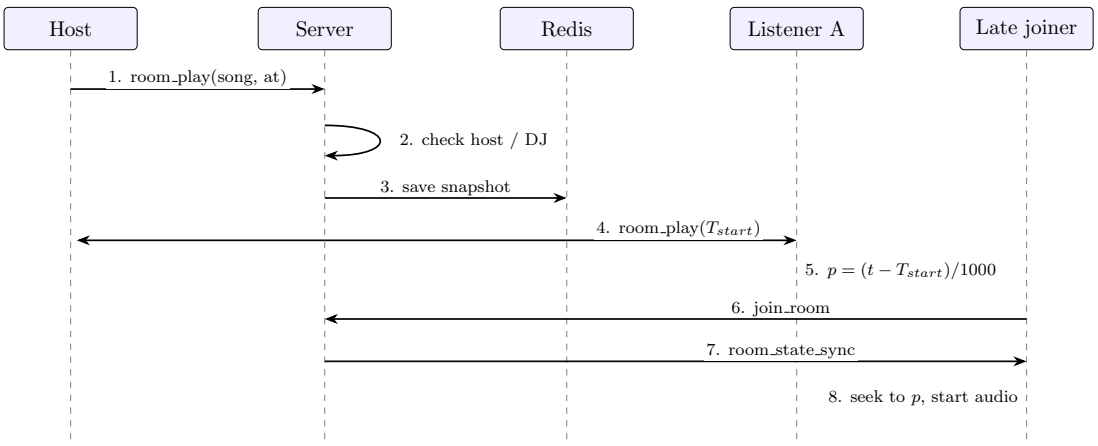


Figure 3.3: Message flow for a host play action and for a late join

Late Join, Reconnection and Drift

A joining participant receives the complete state (current song, playing flag, T_{start}, queue, DJ settings, skip-vote count), seeks to $p(t)$ and starts immediately. If the connection drops, the Socket.IO client reconnects and re-joins, which triggers the same replay; after a server restart the state is restored from its Redis snapshot. The formulas assume that client and server clocks agree. The planned improvement is to estimate the offset θ with an NTP-style ping exchange [2, 3], use $p(t) = (t + \theta - T_{start})/1000$, and seek only when $|p(t) - \text{audio.currentTime}|$ exceeds a drift threshold δ, which avoids audible jumps from tiny differences.

6. Development Plan

The work was organised into the six phases of Table 3.2. Core listening and the real-time layer came first because everything else depends on them.

Table 3.2: Development phases and current status

Phase	Goal	Main scope	Status
1	Foundation	UI shell, backend server, MongoDB, audio streaming, login and sign-up	Completed
2	Core features	Playlists, liked songs, search, UI polish	Completed
3	Rooms	Create rooms, real-time sync, location filtering	Completed
4	Experience	Lyrics support, notifications, themes; mood-aware UI planned	Partially completed
5	Advanced	Artist clips, visual themes; voice control planned	Partially completed
6	Optimization	Redis caching, rate limiting, offline downloads, containers; load tests planned	Partially completed

7. Planned Enhancements

7.1 Enhanced Search

The current search matches song title and artist name case-insensitively and returns up to twenty results. The plan is ranked, typo-tolerant search: text indexes on title, artist, genre and tags, escaping of user input before it is used in a pattern, debounced client requests and Redis caching of popular queries. The search history that is already stored will drive recent and suggested queries.

7.2 Music Recommendations

Plays, saves, likes, recently played tracks and room activity are already recorded. They will feed a first stage of popularity and content-based ranking per genre and language, and later collaborative filtering. A “recommended rooms” list will combine proximity, current activity and the listener’s genres.

7.3 Performance Improvements

Clock-offset compensation and a drift threshold for rooms (Section 3.5); BullMQ workers for e-mail and clean-up; the Socket.IO Redis adapter so the gateway can run on several instances; adaptive audio quality through Cloudinary transformations; measured load tests; and further interaction features such as voice commands, mood-driven themes and shareable room invitation links.

Chapter 4

Implementation

This chapter documents how the planned solution was implemented. The source is organised in two folders, `Frontend` (React application) and `Backend` (Express server and Socket.IO gateway), and is written entirely in TypeScript.

1. Technology Stack

Table 4.1: Technology stack

Layer	Technology	Role
Frontend	React 19, TypeScript, Vite, Tailwind CSS 4, React Router 7, dnd-kit	Single-page application, styling, routing, drag-and-drop [15]
Backend	Node.js, Express 5, TypeScript	REST API and middleware [16]
Real-time	Socket.IO 4 (server and client)	Rooms, broadcast, reconnection [11]
Database	MongoDB with Mongoose	Durable documents, geospatial index [12]
Cache and state	Redis (ioredis)	Cache, room snapshots, presence, rate limits [13]
Media	Cloudinary, Multer, music-metadata	Upload, storage, delivery, duration detection [14]
Security	bcryptjs, jsonwebtoken, Zod, express-rate-limit	Hashing, tokens, validation, throttling
Mail and jobs	Nodemailer, BullMQ	One-time codes by e-mail, job layer
Packaging	Docker, Nginx, GitHub Actions	Images, static hosting, CI/CD [17]

2. Frontend

2.1 Structure, Player and Contexts

The frontend uses a layered layout: `pages` (route-level screens grouped by feature: home, auth, room, artist, library and search, notifications), `components` (global player, route guards), `context`, `api` (one service module per backend resource), `hooks` and `utils`. Protected routes redirect unauthenticated visitors to the login page, and a role check separates listener pages from artist pages. Three contexts hold the cross-cutting state:

- **PlayerContext** creates a single HTML `Audio` element above the router and exposes `play`, `pause`, `seek` and `playSong`, so playback continues while the user navigates. It also records recently played songs.
- **SocketContext** creates one module-level Socket.IO client with the user's token in the handshake, unlimited automatic reconnection and a two-second delay. All room pages share this connection.
- **ThemeContext** stores the selected visual theme, changed through a theme picker.

2.2 Live Session Page

The live-session page is the largest screen. On entry it loads the room through REST and emits `join_room`. Handlers update the player on `room_state_sync`, `room_play`, `room_pause`, `room_seek` and `room_playback_sync`. When a message carries a start time, the page computes $(Date.now() - startTime)/1000$ and seeks the audio element to it, which is the client side of the protocol in Section 3.5; if the browser blocks autoplay, a prompt asks the listener to click once to join the room's audio. For the host, the transport controls call the sync helpers (`syncPlay`, `syncPause`, `syncSeek`, `syncNext`) which emit the matching events; for other participants the controls are disabled, but the real protection is on the server (Section 4.6). The page also shows participants, chat, reactions and the room queue.

2.3 Artist Workspace and Offline Use

Artists see a separate shell with a dashboard, a songs page for uploading and editing tracks, a playlist page, a clips page for short audio posts and an analytics page for plays and saves. Listeners can download songs: the audio is fetched once, stored as a `Blob` in the browser's IndexedDB keyed by song id, and played from there when offline. A session-timeout hook warns after five minutes of inactivity and logs the user out two minutes later, and songs inside a playlist are re-ordered by drag and drop.

3. Backend and Database

The entry file creates an HTTP server from the Express app, attaches the Socket.IO gateway to it, connects to MongoDB and Redis and mounts five routers under `/api`: `auth`, `songs`, `playlists`, `rooms` and `notifications`. Middleware provides CORS limited to the configured frontend origins, body and cookie parsing, JWT protection and multipart upload handling. Routes only wire paths to controllers; controllers hold the business logic; models define the schemas. Table 4.2 lists the main endpoints.

Table 4.2: Main REST endpoints (all under `/api`)

Endpoint	Access	Purpose
POST `/auth/signup`, `/login`, `/google`	Public, rate limited	Create account, log in, Google sign-in
GET `/auth/me`; POST `/auth/2fa/...`	Protected	Current user; two-factor management
POST `/auth/forgot-password`, `/reset-password`	Public, rate limited	Password reset by one-time code
GET `/songs/search`, `/trending`, `/mine`	Protected	Search, cached trending, artist's songs
GET, POST `/playlists`; POST `/:id/songs`	Protected	List, create, add a song
GET, POST `/rooms`; POST `/:id/join`, `/leave`	GET public	List (optional location filter), create, join, leave
POST `/rooms/:id/queue`; PUT `/:id/sync`	Protected	Add to queue, persist playback state
POST `/rooms/:id/dj/toggle`, `/dj/assign`	Protected (host)	Control DJ mode

Data models. The schemas follow Section 3.4. The user model stores profile fields, a role (`user`, `artist` or `admin`), liked and recently played identifiers and the two-factor fields. The

song model stores title, artist, cover and audio URLs with their Cloudinary identifiers, duration, language, lyrics and counters for plays, saves and shares. The playlist model stores owner, visibility, cover image, ordered songs and totals. The room model stores host, participants, current song, queue, settings (chat, queue adding, guests, DJ mode), skip votes, analytics counters and a GeoJSON location with a `2dsphere` index. Passwords and one-time codes are excluded from queries by default.

Validation, rate limiting and location. Request bodies are validated with Zod before any database access; for example, a username must be 3–30 letters, digits or underscores. Authentication routes use `express-rate-limit` with a Redis-backed store, so limits are shared across instances, keyed by the e-mail or username in the request and falling back to the client IP. Room discovery uses a MongoDB `$near` query on the `2dsphere` index: when the client sends coordinates, only public rooms within one kilometre are returned, ordered by whether they are live and by trending score.

4. Authentication and Security

4.1 Sessions and Passwords

On sign-up the password is hashed with bcrypt (10 salt rounds) [6]. On login a JWT [4] containing the user id is signed with a server secret and expires after seven days by default. It is returned in an `httpOnly` cookie (`secure` in production, `SameSite=Lax`) and may also be sent as an `Authorization: Bearer` header. Listing 4.1 shows the middleware that protects routes. The same token is sent in the Socket.IO handshake, where the server verifies it and attaches the user's name, avatar and role to the socket; connections without a valid token may connect as guests but cannot join room state or issue control events.

Listing 4.1: Route-protection middleware (simplified)

```
export const protect = async (req, res, next) => {
  let token;
  if (req.headers.authorization?.startsWith("Bearer")) {
    token = req.headers.authorization.split(" ")[1];
  } else if (req.cookies?.token) {
    token = req.cookies.token;
  }
  if (!token) return res.status(401).json({ success: false, message: "Not authorized" });
  try {
    const decoded = jwt.verify(token, process.env.JWT_SECRET);
    req.user = decoded.id;   // attach user id
    next();
  } catch {
    res.status(401).json({ success: false, message: "Invalid token" });
  }
};
```

4.2 Google Sign-In, Two-Factor Authentication and Recovery

Google sign-in [5] runs in the browser; the backend receives the resulting profile (e-mail, name, avatar, Google id), checks that the account type matches the existing account's role, and links the profile to an account with the same e-mail or creates a new one. Verifying Google's ID token on the server is listed under future work. For two-factor authentication the server generates a six-digit code, stores only its bcrypt hash with a ten-minute expiry and sends it by e-mail; after verification it issues backup codes. Password recovery works the same way. Every one of these endpoints has its own rate limiter.

5. Cloud Media Storage

Audio and images are never written to the application server's disk. The upload middleware uses Multer with in-memory storage, a 25 MB limit and a MIME whitelist (JPEG, PNG and WebP images; MP3, WAV and M4A audio). The song controller then (i) reads the duration from the buffer with `music-metadata`, (ii) streams the buffer to Cloudinary with `upload_stream`, choosing the resource type for audio or image, and (iii) saves the returned URL and public identifier in the song document, updating the playlist's track count and duration if a playlist was given. Finally the trending cache is cleared so new data becomes visible (Figure 4.1).

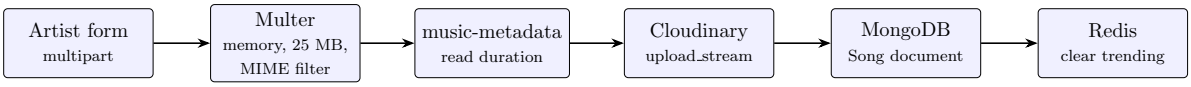


Figure 4.1: Song upload pipeline

6. Real-Time Features

For each active room the gateway keeps an in-memory state object: current song, playing flag, position, T_{start}, queue, participants (keyed by socket id), host id, DJ mode and current DJ, chat and queue settings, recent reactions and the set of users who voted to skip. One function decides who may control playback, and every control event calls it first:

Listing 4.2: Server-side check for playback control

```
const canControlPlayback = (roomState, userId) =>
  roomState.djMode ? roomState.currentDJ === userId
                   : roomState.hostId === userId;
```

Listing 4.3 shows the core of the play handler: the requested position becomes T_{start}, the new state is broadcast to every socket in the room, and it is persisted twice, as a fast Redis snapshot and as the current song in MongoDB. A non-host sender only receives `room_error`.

Listing 4.3: Play handler (simplified)

```
socket.on("room_play", async ({ roomId, at, song }) => {
  const state = getRoomState(roomId);
  if (!state || !userId) return;
  if (!canControlPlayback(state, userId)) {
    socket.emit("room_error", { message: "Only the host or DJ can control playback" });
    return;
  }
  if (song) { state.currentSong = song; state.skipVotes.clear(); }
  state.isPlaying   = true;
  state.currentTime = at ?? state.currentTime;
  state.startTime   = Date.now() - state.currentTime * 1000;   // T_start
  io.to(roomId).emit("room_play", { roomId, at: state.currentTime,
    currentSong: state.currentSong, startTime: state.startTime });
  void persistRoomState(roomId, state);          // Redis snapshot
  Room.findByIdAndUpdate(roomId, { currentSong: state.currentSong, isLive: true });
});
```

Table 4.3 lists the events and the rule the server enforces for each. Each participant's socket sends a presence heartbeat to Redis every 30 seconds with a 90-second time to live, and a clean-up task runs every minute, so crashed or disconnected clients do not leave ghost participants. When the first participant connects after a restart, the room state is rebuilt from

Table 4.3: Socket.IO events and server-side rules

Event	Behaviour
`join_room`	Create or restore room state, register the participant and presence heartbeat, send `room_state_sync` to the newcomer, broadcast the participant list.
`leave_room`, disconnect	Remove participant and presence keys; delete state when the room is empty; if the host leaves, promote the next participant and broadcast `room_host_changed`.
`room_play`, `room_pause`, `room_seek`	Host or DJ only; update state and T_{start}, broadcast, snapshot to Redis, save current song to MongoDB.
`room_next`	Manual skips by host or DJ only; automatic skips are accepted from listeners but ignored if the finished track id does not match, which prevents duplicates.
`room_add_to_queue`, `room_reorder_queue`, `room_remove_from_queue`	Honour the room's queue setting; broadcast the queue and refresh its Redis snapshot.
`room_vote_skip`	Votes are stored per user and cleared on song change; host or DJ skips at once, otherwise skipping happens when votes reach half of the non-host participants (rounded up).
`room_message`, `room_reaction`	Chat is rejected if disabled (except for the host); newest 50 messages kept in the room document; last 30 reactions kept in state.
`room_toggle_dj`, `room_assign_dj`	Host only; changes who may control playback.

the Redis snapshot, then from the queue snapshot, and finally from the MongoDB document.

7. Caching and Performance

Redis is a fast, ephemeral layer in front of MongoDB and around the live session (Table 4.4). Its helper functions are written to be tolerant: when no connection is available they return empty results and the application falls back to MongoDB, so caching improves speed without being required for correctness. No latency figures are claimed here because systematic load measurement is part of the planned work.

Table 4.4: Redis usage

Data	TTL	Purpose and invalidation
Trending songs	5 minutes	Avoid repeated sorting by plays; cleared when song data changes
Room state snapshot	1 hour	Restore a room after restart or reconnect; refreshed on every change
Room queue (sorted set)	1 hour	Fast queue recovery; rewritten when the queue changes
Room presence keys	90 seconds	Detect vanished sockets; renewed by heartbeat
Room detail cache	30 seconds	Reduce repeated reads when many users open one room
Rate-limit counters	per route	Limits shared across instances

8. Deployment and Maintenance

Both applications have multi-stage Dockerfiles. The backend image compiles TypeScript in a builder stage and runs only the compiled output with production dependencies on Node 20 Alpine, exposing port 4000. The frontend image builds the static bundle and serves it with Nginx, with a fallback to `index.html` for client-side routes and an optional proxy of `/api` to the backend; for static hosting the frontend also has a rewrite rule that sends all paths to `index.html`. Secrets and endpoints are supplied through environment variables (database and Redis URLs, JWT secret and expiry, Cloudinary credentials, SMTP settings, Google client identifiers and allowed frontend origins). GitHub Actions workflows build the Node.js backend and deploy it to an Azure Web App on every push to the main branch; a second workflow that deploys over SSH to a virtual machine is kept disabled until needed. ESLint and strict TypeScript settings maintain code quality, the server logs connection, authentication and room events, and the repository documents the product scope, the frontend design hand-off and the setup steps.

9. Verification and Test Scenarios

The scenarios in Table 4.5 verify the behaviour that is specific to Dhuno. Standard create, read and update paths are exercised through the user interface during development; automated tests for these scenarios are listed as future work.

Table 4.5: Verification scenarios for the real-time and security features

ID	Scenario	Expected behaviour
T1	Host plays a song with two listeners connected	Both listeners start the same song within a short interval
T2	A listener emits `room_play` directly	Server replies with `room_error`; state unchanged
T3	A listener joins ten seconds after the song started	Listener seeks to the current position on joining
T4	A listener's connection is interrupted and restored	Socket reconnects, re-joins and receives `room_state_sync`
T5	Host disconnects with others present	Another participant becomes host; `room_host_changed` is broadcast
T6	Host enables DJ mode and assigns a DJ	Only the DJ controls playback until DJ mode is turned off
T7	Upload of a non-audio file or a file above 25 MB	Upload is rejected before reaching storage
T8	Repeated wrong logins from one account	Rate limiter returns an error after the configured limit
T9	Request to a protected route without a token	Response is 401

Chapter 5

Conclusion and Future Work

1. Conclusion

This project set out to build a music streaming platform in which people can listen together in real time and in which artists can publish their own music. The resulting system, Dhuno, achieves this with a React and TypeScript frontend, an Express and Socket.IO backend, MongoDB for durable data, Redis for ephemeral state and Cloudinary for media.

The central contribution is the room model. Instead of broadcasting the playback position continuously, the server stores one reference instant per playback change and every client derives its own position from it. Control is restricted to the host, or to a DJ the host delegates, and this rule is enforced by the server for every control event. Late joiners and reconnecting clients receive a full state replay, and the room state survives restarts through Redis snapshots. Around the rooms, the platform provides the everyday features expected from a streaming service (playlists, likes, search, recently played, offline downloads), an artist workspace, and a security layer with hashed passwords, token sessions, Google sign-in, two-factor authentication and rate limiting.

The work shows that a modest, well-structured MERN application can deliver a shared listening experience that is otherwise available only inside a few commercial ecosystems, and it gave practical experience across the full stack: data modelling, transient versus durable state, safe uploads, authentication hardening and containerised deployment.

2. Future Work

The following improvements are planned, in approximate order of priority.

- **Clock-offset compensation.** Estimate each client's offset from the server with an NTP-style handshake and correct playback only when drift exceeds a tuned threshold (Section 3.5).
- **Security hardening.** Verify Google ID tokens on the server instead of trusting the profile sent by the client, hash the two-factor backup codes, and restrict cross-origin headers on the static host to the known frontend origin.
- **Enhanced search.** Replace pattern matching with indexed, typo-tolerant, ranked search, escape user input before use, and cache popular queries.
- **Recommendations.** Build content-based and popularity-based suggestions from plays, likes and room activity, followed by collaborative filtering and a recommended-rooms list.
- **Horizontal scaling.** Use the Socket.IO Redis adapter and move deferred work such as e-mail and clean-up to BullMQ workers so that several backend instances can share the load.
- **Automated testing and measurement.** Add unit and integration tests for the scenarios in Table 4.5, end-to-end tests with several simulated clients, and load tests that measure synchronization error and response times.
- **Richer interaction.** Voice commands, mood-aware themes, room invitation links and activity feeds.

Bibliography

[1] I. Fette and A. Melnikov, "The WebSocket Protocol," RFC 6455, Internet Engineering Task Force, Dec. 2011.

[2] D. Mills, J. Martin, J. Burbank and W. Kasch, "Network Time Protocol Version 4: Protocol and Algorithms Specification," RFC 5905, Internet Engineering Task Force, Jun. 2010.

[3] F. Cristian, "Probabilistic clock synchronization," *Distributed Computing*, vol. 3, no. 3, pp. 146–158, 1989.

[4] M. Jones, J. Bradley and N. Sakimura, "JSON Web Token (JWT)," RFC 7519, Internet Engineering Task Force, May 2015.

[5] D. Hardt (ed.), "The OAuth 2.0 Authorization Framework," RFC 6749, Internet Engineering Task Force, Oct. 2012.

[6] N. Provos and D. Mazières, "A future-adaptable password scheme," in *Proc. USENIX Annual Technical Conference, FREENIX Track*, 1999, pp. 81–91.

[7] R. T. Fielding, "Architectural styles and the design of network-based software architectures," Ph.D. dissertation, University of California, Irvine, 2000.

[8] "Spotify launches Jam, a real-time collaborative playlist controlled by up to 32 people," *TechCrunch*, Sep. 26, 2023. [Online]. Available: https://techcrunch.com/2023/09/26/spotify-launches-jam-a-real-time-collaborative-playlist-controlled-by-up-to-32-people

[9] Apple Inc., "SharePlay powers new ways to stay connected and share experiences in FaceTime," Apple Newsroom, Nov. 2021. [Online]. Available: https://www.apple.com/newsroom/2021/11/shareplay-powers-new-ways-to-stay-connected-and-share-experiences-in-facetime

[10] "Turntable.fm," Wikipedia. [Online]. Available: https://en.wikipedia.org/wiki/Turntable.fm

[11] Socket.IO Documentation, v4. [Online]. Available: https://socket.io/docs/v4/

[12] MongoDB Inc., "2dsphere indexes," MongoDB Manual. [Online]. Available: https://www.mongodb.com/docs/manual/core/2dsphere/

[13] Redis Documentation. [Online]. Available: https://redis.io/docs/

[14] Cloudinary Documentation. [Online]. Available: https://cloudinary.com/documentation

[15] React Documentation. [Online]. Available: https://react.dev/

[16] Express.js Documentation. [Online]. Available: https://expressjs.com/

[17] Docker Documentation. [Online]. Available: https://docs.docker.com/